Parallel v3.2 Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Stalin

Alix40

T1MOH

May 24, 2026

Contents

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Parallel Protocol v3.2 is an upgrade to Parallel v3.1 that introduces `EIP-3009` gasless authorization flows, allowing relayers to execute swaps, redemptions, savings operations, and USDP interactions on behalf of users. The release also includes a hotfix that patches the ability to put the protocol into a zero-debt state.

5 Audit Scope

The audit scope covers modifications across two repositories to support the introduction of EIP-3009 and a hotfix that patches the ability to put the protocol into a zero-debt state.

The changes for the Parallel-Tokens repo are in PR 15:

```
Parallel-Tokens at commit hash `627d8b869fd11ebef517596328dbb4106f28b018`:
* EIP3009.sol
* IEIP3009.sol
* TokenP.sol
```

The changes for the Parallel-Parallelizer repo are in PR 21:

```
Parallel-Parallelizer at commit hash `a4870476078fdcd48ed77a247b0e806476f9b196`:
* EIP3009.sol
* IEIP3009.sol
* Swapper.sol
* ISwapper.sol
* Redeemer.sol
* IRedeemer.sol
* LibAuthorization.sol
* Storage.sol
* Savings.sol
* ISavings.sol
* SavingsEIP3009.sol
```

```
* Errors.sol
```

6 Executive Summary

Over the course of 5 days, the Cyfrin team conducted an audit on the Parallel v3.2 code provided by Parallel. The findings consist of 1 Low severity issue with the remainder being informational.

6.1 Centralization Risks

The protocol is governed by a single AccessManager governor role with broad administrative powers. Users must trust governance to operate the system responsibly.

Inherent Admin Powers:

- The governor can mint USDp to any address and burn USDp from any account via `TokenP::mint` and `TokenP::burnFrom`, giving it direct control over the circulating supply.
- The governor can pause the Savings vault indefinitely via `Savings::togglePause`, blocking all deposits and withdrawals with no time-bound or user-accessible emergency exit path.
- Both `TokenP` and `Savings` are UUPS upgradeable proxies. The governor can upgrade either implementation at will, replacing all contract logic.
- The governor controls the savings yield rate (via `Savings::setRate` and `Savings::setMaxRate`) with no on-chain ceiling above `maxRate`, which is itself governor-set. There is no code-level guard preventing a rate that outpaces actual protocol surplus generation.
- The governor can add and remove trusted rate-setting addresses via `Savings::toggleTrusted`, indirectly controlling who can update the yield rate.

Upstream Trust Assumptions:

- The Parallelizer's collateralization ratio depends on governance or a trusted keeper periodically calling `Redeemer::updateNormalizer` to reconcile yield minted by the Savings vault. Between reconciliation calls, the reported collateral ratio overstates actual backing, creating a window where early redeemers receive more collateral per USDp than is mathematically justified.

6.2 Post Audit Recommendations

During the audit, an issue was discovered in the deployed implementation of the Savings contract. As a result, the Savings contract has been paused across all chains and will remain paused until a fix is proposed, audited, and deployed to upgrade the current implementation. As part of the mitigation, stale interest was forcibly accrued via a call to `setRate`. Although the contract remains paused, we recommend calling `setRate` on a recurring basis to prevent any unforeseen side effects when performing the upgrade with a stale accrual window.

Summary

Project Name	Parallel v3.2
Repository	parallel-tokens
Commit	627d8b869fd1...
Fix Commit	ed47f03d4e66...
Repository 2	parallel-parallelizer
Commit	a4870476078f...
Fix Commit	671e1e89d62f...
Audit Timeline	May 5th - May 11th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	1
Informational	7
Gas Optimizations	0
Total Issues	8

Summary of Findings

[L-1] `Savings::initialize` does not validate divizer, enabling first-depositor inflation attack	Resolved
[I-1] `Savings::setMaxRate` does not clamp the active rate below the new cap	Resolved
[I-2] `Savings::togglePause` defers yield accrual across the pause window rather than freezing it	Resolved
[I-3] Savings max-view functions do not return 0 when paused, breaking ERC4626 compliance	Resolved
[I-4] `ISavings::initializeEIP3009` is declared in the interface but never implemented on Savings	Resolved
[I-5] TokenP constructor uses OZ v4 `initializer` rather than `_disableInitializers`	Resolved
[I-6] `Redeemer::_quoteRedemptionCurve` evaluates `penaltyFactor` at entry-time CR on the full `amountBurnt`, over-penalizing large single redemptions that span into lower-penalty curve segments	Resolved
[I-7] `TokenP` dual inheritance from `ERC20PermitUpgradeable` and `EIP3009` creates independent EIP-712 version sources - `VERSION_HASH` constant diverges from storage on a version-bumping upgrade, breaking `permit()`	Resolved

7 Findings

7.1 Low Risk

7.1.1 `Savings::initialize` does not validate divizer, enabling first-depositor inflation attack

Description: `Savings::initialize` seeds the vault dead-share moat using:

```
_deposit(msg.sender, address(this), 10 ** (asset_.decimals()) / divizer, BASE_18 / divizer);
```

There is no `require` bounding `divizer` above zero or below `min(10**asset.decimals, BASE_18)`. Solidity 0.8 reverts for `divizer == 0` (division by zero), but any `divizer > BASE_18` or `> 10**asset.decimals` silently floors both operands to `0`. OpenZeppelin's `_deposit` accepts zero assets and zero shares without reverting, leaving `totalSupply == 0` and `totalAssets == 0` -- disabling the dead-share moat that protects against the classical ERC-4626 first-depositor inflation attack.

Additionally, `initialize` accepts empty `name_` or `symbol_` strings with no validation. An empty `name_` propagates into the EIP-712 domain separator computed in `EIP3009::_domainSeparator` via `keccak256(bytes(name()))`, producing a domain whose wallet-display name is blank. This degrades signing UX for every `depositWithAuthorization` and `redeemWithAuthorization` user.

Affected location: `parallel-parallelizer/contracts/savings/Savings.sol:63-80`

Impact: On any deployment where the admin passes `divizer > 10**asset.decimals` (e.g., `divizer = 1e18 + 1` for a D18 asset), the first-depositor protection is silently disabled. An attacker can then mint one share, donate a large amount of USDp directly to the vault, and dilute the next legitimate depositor to zero shares -- draining their full deposit. The deploy script currently hardcodes `initialDivider = "1"`, so production is not immediately exposed, but the on-chain `initialize` accepts any value with no guardrail, leaving future deployments and re-initializations at risk.

Recommended Mitigation: Add input validation in `Savings::initialize` before calling `_deposit`:

```
if (divizer == 0 || BASE_18 / divizer == 0 || 10 ** asset_.decimals() / divizer == 0)
    revert InvalidParam();
if (bytes(name_).length == 0 || bytes(symbol_).length == 0)
    revert InvalidParam();
```

Optionally also assert `totalSupply > 0` after the seed deposit to confirm the dead-share moat was planted.

Parallel: Fixed in commit b4a854a.

Cyfrin: Verified. `name`, `symbol` and `divizer` are now validated not to be 0.

7.2 Informational

7.2.1 `Savings::setMaxRate` does not clamp the active rate below the new cap

Description: `Savings::setMaxRate` overwrites `maxRate` storage and emits an event. It does not (a) call `_accrue` to settle outstanding interest at the prior rate, and does not (b) clamp `rate = min(rate, newMaxRate)`. The NatSpec at line 32-34 acknowledges this: "Note that `rate` can still be greater than `maxRate` if this `maxRate` is reduced by governance to a level inferior to the current rate." The companion setter `setRate` DOES call `_accrue` first and enforces `newRate <= maxRate` at call time, but only for that one path. Reducing `maxRate` below the current `rate` does not trigger the same enforcement.

Affected location: `parallel-parallelizer/contracts/savings/Savings.sol:404-407`

Impact: After a governance reduction of `maxRate`, accrual continues at the old (now-exceeding-cap) rate until a separate `setRate` transaction lands. The excess accrual mints `(rate - newMaxRate) * elapsed / BASE_27 * totalAssets` of TokenP per second beyond the new cap. Every second of uncapped accrual also widens the normalizer-drift gap between reported and actual stablecoin supply. The asymmetric handling between the two range-defining setters creates a latent invariant violation: `rate <= maxRate`, which `setRate` enforces, is silently broken whenever `setMaxRate` shrinks `maxRate`.

Recommended Mitigation: In `Savings::setMaxRate`, call `_accrue` first to settle yield at the old rate, then clamp the active rate:

```
function setMaxRate(uint256 newMaxRate) external restricted {
    _accrue();
    maxRate = newMaxRate;
    if (rate > newMaxRate) {
        rate = uint208(newMaxRate);
        emit RateUpdated(newMaxRate);
    }
    emit MaxRateUpdated(newMaxRate);
}
```

This makes `setMaxRate` self-sufficient -- a single transaction enforces the new cap without requiring a follow-up `setRate` call.

Parallel: Fixed in commit d3e4824.

Cyfrin: Verified. When updating `maxRate`, the outstanding yield at the prior rate is settled before mutating `maxRate`, and the active `rate` is clamped down to the new cap when it would otherwise exceed it.

7.2.2 `Savings::togglePause` defers yield accrual across the pause window rather than freezing it

Description: `Savings` is an ERC-4626 vault that accrues compound interest by calling `Savings::_accrue` at the start of every state-changing interaction. `Savings::_accrue` reads the current block timestamp, computes the interest earned since `lastUpdate`, mints that delta to the vault, and advances `lastUpdate` to the present:

```
// Savings.sol:104-114
function _accrue() internal returns (uint256 newTotalAssets) {
    uint256 currentBalance = super.totalAssets();
    newTotalAssets = _computeUpdatedAssets(currentBalance, block.timestamp - lastUpdate);
    lastUpdate = uint40(block.timestamp);
    uint256 earned = newTotalAssets - currentBalance;
    if (earned > 0) {
        ITokenP(asset()).mint(address(this), earned);
    }
}
```

Because `Savings::togglePause` does not call `_accrue` on the PAUSE transition, `lastUpdate` remains pinned at the timestamp of the last pre-pause interaction. During the pause, no interaction can advance `lastUpdate`. When the vault is unpaused, `lastUpdate` is stale by the entire pause duration.

Recommended Mitigation: Call `Savings::_accrue` inside `Savings::togglePause` before the `paused` flag is changed, covering both transitions:

```
function togglePause() external restricted {
    _accrue();
    uint8 pauseStatus = 1 - paused;
    paused = pauseStatus;
    emit ToggledPause(pauseStatus);
}
```

Calling `Savings::_accrue` on the PAUSE transition snapshots `lastUpdate` to the pause timestamp, so accumulated interest is distributed to current holders at the moment the vault stops accepting interactions. Calling it on the UNPAUSE transition advances `lastUpdate` to the unpause timestamp, eliminating the stale gap that would otherwise persist until the first user interaction.

Parallel: Fixed in commit 671e1e8.

Cyfrin: Verified. Outstanding yield is now accrued on `togglePause` transitions to remove stale-window deferral.

7.2.3 Savings max-view functions do not return 0 when paused, breaking ERC4626 compliance

Description: `Savings::deposit`, `mint`, `withdraw`, and `redeem` all carry the `whenNotPaused` modifier. The contract does not override `maxDeposit`, `maxMint`, `maxWithdraw`, or `maxRedeem`. The OpenZeppelin ERC4626 defaults return `type(uint256).max` for the deposit/mint pair and asset-denominated balance values for the withdraw/redeem pair, regardless of pause state.

ERC4626 (EIP-4626) requires: "MUST factor in both global and user-specific limits, like if deposits are entirely disabled (even temporarily) it MUST return 0." When `paused > 0`, all four call paths revert, but the views continue to advertise non-zero capacity.

Affected location: `parallel-parallelizer/contracts/savings/Savings.sol:144-187`

Impact: Any integrator that follows the ERC4626 spec and uses `max*` to gate calls will fail when Savings is paused. Routers, meta-vaults, and yield aggregators built on the ERC4626 contract surface receive incorrect guidance from the `max*` views and subsequently revert on integration paths that the spec promises will succeed.

Recommended Mitigation: Override the four `max*` views in `Savings`:

```
function maxDeposit(address) public view override returns (uint256) {
    return paused > 0 ? 0 : type(uint256).max;
}
function maxMint(address) public view override returns (uint256) {
    return paused > 0 ? 0 : type(uint256).max;
}
function maxWithdraw(address owner) public view override returns (uint256) {
    return paused > 0 ? 0 : super.maxWithdraw(owner);
}
function maxRedeem(address owner) public view override returns (uint256) {
    return paused > 0 ? 0 : super.maxRedeem(owner);
}
```

Parallel: Fixed in commit d191c62.

Cyfrin: Verified. max-functions now returns 0 when the `Savings` contract is paused.

7.2.4 `ISavings::initializeEIP3009` is declared in the interface but never implemented on Savings

Description: `ISavings` declares `initializeEIP3009(string memory name_)` at line 77, but no contract in the `Savings` inheritance chain (BaseSavings, SavingsEIP3009, EIP3009) implements it. The on-chain `Savings` contract does not have this selector registered. Any integrator who reads the interface and assumes a post-deployment EIP-3009 initialization step is required will encounter a revert on deployment. The actual implementation does not

need this hook because `EIP3009::_domainSeparator` is computed dynamically from `name` and requires no bootstrapping call.

The asymmetry -- `initialize` is implemented and required, but `initializeEIP3009` is declared but absent -- is the surprise vector for developers wiring up the Savings ABI.

Affected location: `parallel-parallelizer/contracts/interfaces/ISavings.sol:77`

Recommended Mitigation: Remove `function initializeEIP3009(string memory name_) external;` from `ISavings.sol`. The dynamic domain separator implementation confirms no init hook is needed; the orphan interface declaration is the artifact to drop.

Parallel: Fixed in commit a11c8bf.

Cyfrin: Verified. The orphan `initializeEIP3009` declaration has been removed from the `ISavings` interface.

7.2.5 TokenP constructor uses OZ v4 `initializer` rather than `_disableInitializers`

Description: The `TokenP` UUPS implementation uses `constructor() initializer {}` (TokenP.sol:34) instead of the OZ v5.4 pattern `constructor() { _disableInitializers(); }` that the sibling `BaseSavings` contract already adopts (BaseSavings.sol:35-37). In OZ v5.4 `Initializable`, the `initializer` modifier sets `$._initialized = 1`, blocking only the version-1 `initialize` call on the implementation contract. It does NOT prevent any future `reinitializer(N)` for `N > 1` because `_initialized = 1 < type(uint64).max`. By contrast, `_disableInitializers` sets `$._initialized = type(uint64).max`, permanently locking every initializer and reinitializer entry point on the implementation address.

No `reinitializer` function currently exists in `TokenP.sol`, so the dangling surface has no callable entry today. The risk is forward-looking: any future implementation that adds a `reinitializer(N)` becomes immediately callable on the implementation address, potentially letting an attacker set the implementation's `authority` and chain an `_authorizeUpgrade` call.

Affected location: `parallel-tokens/contracts/tokens/TokenP/TokenP.sol:33-34`

Recommended Mitigation: Replace `constructor() initializer {}` with `constructor() { _disableInitializers(); }`, matching `BaseSavings.sol:36`. Keep the `/// @custom:oz-upgrades-unsafe-allow constructor` annotation. After the change, `$._initialized` on the implementation is set to `type(uint64).max`, permanently blocking all current and future `initializer`/`reinitializer(N)` entry points on the implementation contract address.

Parallel: Fixed in commit f616390.

Cyfrin: Verified. `TokenP` constructor now uses `_disableInitializers` instead of `initializer`.

7.2.6 `Redeemer::_quoteRedemptionCurve` evaluates `penaltyFactor` at entry-time CR on the full `amountBurnt`, over-penalizing large single redemptions that span into lower-penalty curve segments

Description: `Redeemer::_quoteRedemptionCurve` snapshots the current `collatRatio` once at call entry, converts it to a `penaltyFactor` via `LibHelpers::piecewiseLinear`, and applies that single rate uniformly to the entire `amountBurnt`:

```
// Redeemer.sol:267-296
penaltyFactor = uint64(LibHelpers.piecewiseLinear(collatRatio, xRedemptionCurve, yRedemptionCurve));
...
for (uint256 i; i < tokens.length; ++i) {
    balances[i] = (amountBurnt * balances[i] * penaltyFactor) / (stablecoinsIssued * BASE_9);
}
```

The function does not integrate the penalty across the CR path the redemption would traverse. A large redemption that begins in a high-penalty region but, by its size, would push CR into a lower-penalty region still incurs the full entry-time `penaltyFactor` on every unit burned.

Concrete example using the production curve:

```
Production redemption curve (all deployed chains):
- `xRedemptionCurve = [0.75, 0.85, 0.95, 0.97] × BASE_9`
- `yRedemptionCurve = [0.995, 0.95, 0.95, 0.995] × BASE_9`
```

The ascending segment `CR [0.95, 0.97]` recovers from PF=0.95 back to PF=0.995. Consider a redeemer whose `amountBurnt` is large enough to move CR from 0.95 all the way to 0.97:

- Single call (current behavior): entry CR = 0.95, PF = 0.95 applied to 100% of `amountBurnt`. Effective blended penalty = 5%.
- Infinite-granularity split (economic ideal): each infinitesimal slice is priced at the CR prevailing at that point on the path. Blended PF (0.95 + 0.995) / 2 = 0.9725. Effective blended penalty 2.75%.

The single-call redeemer is over-penalized by ~225 bps relative to the path-integrated rate. The advantage of splitting grows with (a) the slope of the ascending segment, and (b) how much of that segment the redemption traverses.

This is a direct consequence of the design choice to evaluate the penalty curve as a snapshot rather than an integral. The same design is coherent with the protocol's stated goal of penalizing early redeemers during undercollateralization — however, the snapshot approach creates a discontinuity between the price a single-call redeemer pays and the path-integrated price a patient redeemer who splits would pay.

Impact: A discontinuity between the price a single-call redeemer pays and the path-integrated price a patient redeemer who splits would pay.

Proof of Concept:

```solidity
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.28;

// Run from parallel-parallelizer/ directory:
//   forge test --match-contract PocRedemptionOverPenalization -vvv

/// @title  PoC: Redeemer Over-Penalization - Entry-Time Fee Rate Applied to Full amountBurnt
/// @notice Title:    Large single redemptions pay entry-time fee on entire amount; splits benefit from
///                   progressively lower fees as CR improves after each partial burn
///         Affected: parallel-parallelizer/contracts/parallelizer/facets/Redeemer.sol:267-296
///         Impact:   Single-call redeemers pay more in penalty fees than equivalent split redeemers
///                   when the redemption spans the ascending segment of the penalty curve (CR
↪  0.95-0.97)
///         Author:   0xStalin / Cyfrin

import { Fixture } from "../Fixture.sol";
import { MockChainlinkOracle } from "../mock/MockChainlinkOracle.sol";
import "contracts/utils/Constants.sol";
import "contracts/parallelizer/Storage.sol";
import { console } from "@forge-std/console.sol";

// == [ Contract ] ==
contract PocRedemptionOverPenalization is Fixture {

    // == [ Constants ] ==
    // Production redemption curve (all deployed chains).
    // xRedeemFee in BASE_9 units: [0.75, 0.85, 0.95, 0.97]
    // yRedeemFee in BASE_9 units: [0.995, 0.95, 0.95, 0.995]
    // The ascending segment is CR  [0.95, 0.97] - PoC exploits this range.
    uint64 constant X0 = 750_000_000;  // 0.75 × BASE_9
    uint64 constant X1 = 850_000_000;  // 0.85 × BASE_9
    uint64 constant X2 = 950_000_000;  // 0.95 × BASE_9
    uint64 constant X3 = 970_000_000;  // 0.97 × BASE_9

    int64 constant Y0 = 995_000_000;   // 0.995 × BASE_9
    int64 constant Y1 = 950_000_000;   // 0.95  × BASE_9
```

```
    int64 constant Y2 = 950_000_000;   // 0.95  * BASE_9
    int64 constant Y3 = 995_000_000;   // 0.995 * BASE_9

    // Deposit 1,000,000 eurA (6-decimal token)
    uint256 constant DEPOSIT_EURA = 1_000_000e6;

    // Oracle: initial = 1.0 in Chainlink 8-dec format; drop = 0.95
    int256 constant ORACLE_INITIAL  = int256(BASE_8);            // 1e8
    int256 constant ORACLE_DROPPED  = int256(95 * BASE_8 / 100); // 0.95e8

    // tokenP amounts for redemptions (18-decimal)
    uint256 constant REDEEM_TOTAL  = 200_000e18; // single call
    uint256 constant REDEEM_HALF   = 100_000e18; // each of 2 splits
    uint256 constant REDEEM_FIFTH  =  40_000e18; // each of 5 splits

    address internal redeemer;

    // == [ setUp ] ==
    function setUp() public override {
        super.setUp();

        redeemer = vm.addr(10);
        vm.label(redeemer, "Redeemer");

        // -- Set mint fees = 0 for eurA so swapExactInput works cleanly --
        uint64[] memory xFeeMint = new uint64[](1);
        xFeeMint[0] = uint64(0);
        int64[] memory yFeeMint = new int64[](1);
        yFeeMint[0] = 0;

        uint64[] memory xFeeBurn = new uint64[](1);
        xFeeBurn[0] = uint64(BASE_9);
        int64[] memory yFeeBurn = new int64[](1);
        yFeeBurn[0] = 0;

        vm.startPrank(guardian);
        parallelizer.setFees(address(eurA), xFeeMint, yFeeMint, true);  // mint fees = 0
        parallelizer.setFees(address(eurA), xFeeBurn, yFeeBurn, false); // burn fees = 0

        // -- Set production redemption curve --
        uint64[] memory xRedeemFee = new uint64[](4);
        xRedeemFee[0] = X0;
        xRedeemFee[1] = X1;
        xRedeemFee[2] = X2;
        xRedeemFee[3] = X3;

        int64[] memory yRedeemFee = new int64[](4);
        yRedeemFee[0] = Y0;
        yRedeemFee[1] = Y1;
        yRedeemFee[2] = Y2;
        yRedeemFee[3] = Y3;

        parallelizer.setRedemptionCurveParams(xRedeemFee, yRedeemFee);
        vm.stopPrank();

        // -- Redeemer deposits 1,000,000 eurA at oracle = 1.0 → mints ~1M tokenP --
        // oracle is already at BASE_8 (1e8) from Fixture.setUp
        vm.startPrank(redeemer);
        deal(address(eurA), redeemer, DEPOSIT_EURA);
        eurA.approve(address(parallelizer), DEPOSIT_EURA);
        parallelizer.swapExactInput(
            DEPOSIT_EURA,
            0,
```

```
            address(eurA),
            address(tokenP),
            redeemer,
            block.timestamp * 2
        );
        vm.stopPrank();
    }

    // == [ Helper: drop oracle and assert CR in ascending region ] ==
    function _dropOracleAndAssertRegion() internal {
        MockChainlinkOracle(address(oracleA)).setLatestAnswer(ORACLE_DROPPED);

        (uint64 collatRatio,) = parallelizer.getCollateralRatio();
        assertGe(collatRatio, uint64(950_000_000), "CR must be >= 0.95 x BASE_9");
        assertLe(collatRatio, uint64(970_000_000), "CR must be <= 0.97 x BASE_9");
    }

    // == [ Helper: build zero-slippage minAmountOuts for 3 collaterals ] ==
    function _zeroMins() internal pure returns (uint256[] memory mins) {
        mins = new uint256[](3);
        // all zeros - accept any amount
    }

    // == [ Helper: piecewiseLinear penalty factor from CR ] ==
    function _pfFromCR(uint64 cr) internal pure returns (uint64 pf, uint64 feeRateBps) {
        // piecewiseLinear over production curve
        if (cr >= 970_000_000) {
            pf = 995_000_000;
        } else if (cr >= 950_000_000) {
            // ascending segment [0.95, 0.97]: PF goes from 950e6 to 995e6
            pf = uint64(950_000_000 + uint256(cr - 950_000_000) * (995_000_000 - 950_000_000) /
            ↪  (970_000_000 - 950_000_000));
        } else if (cr >= 850_000_000) {
            pf = 950_000_000; // flat minimum
        } else if (cr >= 750_000_000) {
            // descending segment [0.75, 0.85]
            pf = uint64(995_000_000 - uint256(cr - 750_000_000) * (995_000_000 - 950_000_000) /
            ↪  (850_000_000 - 750_000_000));
        } else {
            pf = 995_000_000;
        }
        // fee rate in bps = (BASE_9 - pf) * 10000 / BASE_9
        feeRateBps = uint64((uint256(1_000_000_000 - pf) * 10_000) / 1_000_000_000);
    }

    // == [ Test 1: single call vs N=2 split - single is over-penalized ] ==
    function test_overPenalization_singleVsN2() public {
        _dropOracleAndAssertRegion();

        // == [ Entry state ] ==
        (uint64 entryCR,) = parallelizer.getCollateralRatio();
        (uint64 entryPF, uint64 entryFeeBps) = _pfFromCR(entryCR);

        console.log("[PoC] ==================== ENTRY STATE ====================");
        console.log("[PoC] collat ratio (CR):        ", entryCR);
        console.log("[PoC]                            (0.95 x BASE_9)");
        console.log("[PoC] penalty factor (PF):      ", entryPF);
        console.log("[PoC]                            (0.95 x BASE_9 => 5.00% fee)");

        // Take a snapshot before any redemption
        uint256 snapId = vm.snapshotState();

        // == [ PATH A: single call of REDEEM_TOTAL ] ==
```

```
        console.log("[PoC] ==================== PATH A: SINGLE CALL ====================");

        (uint64 crA,) = parallelizer.getCollateralRatio();
        (uint64 pfA, uint64 feeA) = _pfFromCR(crA);

        console.log("[PoC] CR at entry:              ", crA);
        console.log("[PoC] PF applied to ALL tokens: ", pfA);
        console.log("[PoC] Effective fee rate:       ", feeA);
        console.log("[PoC]                           bps (5.00%)");

        uint256 eurABefore_single = eurA.balanceOf(redeemer);
        vm.prank(redeemer);
        parallelizer.redeem(REDEEM_TOTAL, redeemer, block.timestamp * 2, _zeroMins());
        uint256 singleOut = eurA.balanceOf(redeemer) - eurABefore_single;

        console.log("[PoC] Collateral received (eurA):", singleOut);

        // Revert to pre-redemption state
        vm.revertToState(snapId);

        // == [ PATH B: N=2 split, REDEEM_HALF each ] ==
        console.log("[PoC] ==================== PATH B: SPLIT N=2 ====================");

        uint256 eurABefore_split = eurA.balanceOf(redeemer);

        // Split 1
        (uint64 cr_b1,) = parallelizer.getCollateralRatio();
        (uint64 pf_b1, uint64 fee_b1) = _pfFromCR(cr_b1);
        vm.prank(redeemer);
        parallelizer.redeem(REDEEM_HALF, redeemer, block.timestamp * 2, _zeroMins());
        uint256 received_b1 = eurA.balanceOf(redeemer) - eurABefore_split;

        console.log("[PoC] Split 1: CR=            ", cr_b1);
        console.log("[PoC]           PF=            ", pf_b1);
        console.log("[PoC]           fee=           ", fee_b1);
        console.log("[PoC]           bps => received:", received_b1);
        console.log("[PoC]           eurA");

        // Split 2
        (uint64 cr_b2,) = parallelizer.getCollateralRatio();
        (uint64 pf_b2, uint64 fee_b2) = _pfFromCR(cr_b2);

        uint256 eurABefore_split2 = eurA.balanceOf(redeemer);
        vm.prank(redeemer);
        parallelizer.redeem(REDEEM_HALF, redeemer, block.timestamp * 2, _zeroMins());
        uint256 received_b2 = eurA.balanceOf(redeemer) - eurABefore_split2;

        console.log("[PoC] Split 2: CR=            ", cr_b2);
        console.log("[PoC]           PF=            ", pf_b2);
        console.log("[PoC]           fee=           ", fee_b2);
        console.log("[PoC]           bps => received:", received_b2);
        console.log("[PoC]           eurA  <-- lower PF due to improved CR");

        uint256 splitN2Out = eurA.balanceOf(redeemer) - eurABefore_split;
        console.log("[PoC] Total received:          ", splitN2Out);
        console.log("[PoC]                           eurA");

        // == [ Summary ] ==
        uint256 extraN2 = splitN2Out - singleOut;
        uint256 bpsN2   = (extraN2 * 10_000) / singleOut;

        console.log("[PoC] ==================== SUMMARY: OVER-PENALIZATION (N=2)
        ↪  ====================");
```

```
        console.log("[PoC] Single call output (eurA):        ", singleOut);
        console.log("[PoC] Split N=2 output (eurA):          ", splitN2Out);
        console.log("[PoC] Over-penalization single vs N=2:  ", extraN2);
        console.log("[PoC]   eurA =                          ", bpsN2);
        console.log("[PoC]   bps of single output");

        // == [ Assertions ] ==
        assertGt(splitN2Out, singleOut, "Split N=2 must receive more eurA than single call");
        // Split 2 receives more than split 1 due to two compounding effects:
        // (a) lower penaltyFactor  CR improved into the ascending segment, so PF is higher  smaller
        ↪  haircut
        // (b) ratio uplift  burning 100k tokenP reduced stablecoinsIssued faster than collateral value,
        //     raising the balance/issuance ratio for all subsequent callers
        assertGt(received_b2, received_b1, "Split 2 must receive more than split 1 (improved CR + ratio
        ↪  effect)");
    }

    // == [ Test 2: single call vs N=5 split - fee rate strictly decreases ] ==
    function test_overPenalization_singleVsN5() public {
        _dropOracleAndAssertRegion();

        // == [ Entry state ] ==
        (uint64 entryCR,) = parallelizer.getCollateralRatio();
        (uint64 entryPF,) = _pfFromCR(entryCR);

        console.log("[PoC] ==================== ENTRY STATE ====================");
        console.log("[PoC] collat ratio (CR):        ", entryCR);
        console.log("[PoC]                            (0.95 x BASE_9)");
        console.log("[PoC] penalty factor (PF):      ", entryPF);
        console.log("[PoC]                            (0.95 x BASE_9 => 5.00% fee)");

        // Take a snapshot before any redemption
        uint256 snapId = vm.snapshotState();

        // == [ PATH A: single call ] ==
        console.log("[PoC] ==================== PATH A: SINGLE CALL ====================");

        (uint64 crA,) = parallelizer.getCollateralRatio();
        (, uint64 feeA) = _pfFromCR(crA);

        console.log("[PoC] CR at entry:              ", crA);
        console.log("[PoC] Effective fee rate:       ", feeA);
        console.log("[PoC]                            bps (5.00%)");

        uint256 eurABefore_single = eurA.balanceOf(redeemer);
        vm.prank(redeemer);
        parallelizer.redeem(REDEEM_TOTAL, redeemer, block.timestamp * 2, _zeroMins());
        uint256 singleOut = eurA.balanceOf(redeemer) - eurABefore_single;

        console.log("[PoC] Collateral received (eurA):", singleOut);

        // Revert to pre-redemption state
        vm.revertToState(snapId);

        // == [ PATH C: N=5 split, REDEEM_FIFTH each ] ==
        console.log("[PoC] ==================== PATH C: SPLIT N=5 ====================");

        uint256 eurABefore_n5 = eurA.balanceOf(redeemer);

        uint256[5] memory received_c;
        uint64[5]  memory fee_c;

        for (uint256 i; i < 5; ++i) {
```

```
            (uint64 cr_ci,) = parallelizer.getCollateralRatio();
            (, uint64 fee_ci) = _pfFromCR(cr_ci);
            fee_c[i] = fee_ci;

            uint256 balBefore = eurA.balanceOf(redeemer);
            vm.prank(redeemer);
            parallelizer.redeem(REDEEM_FIFTH, redeemer, block.timestamp * 2, _zeroMins());
            received_c[i] = eurA.balanceOf(redeemer) - balBefore;

            console.log("[PoC] Split ", i + 1);
            console.log("[PoC]   CR=                    ", cr_ci);
            console.log("[PoC]   fee=                   ", fee_ci);
            console.log("[PoC]   bps => received:       ", received_c[i]);
            console.log("[PoC]   eurA");
        }

        uint256 splitN5Out = eurA.balanceOf(redeemer) - eurABefore_n5;
        console.log("[PoC] Total received:            ", splitN5Out);
        console.log("[PoC]                             eurA");

        // Revert again to compare N=2 baseline
        vm.revertToState(snapId);

        // == [ PATH B: N=2 split - reproduced for the 3-way summary ] ==
        uint256 eurABefore_n2 = eurA.balanceOf(redeemer);

        vm.prank(redeemer);
        parallelizer.redeem(REDEEM_HALF, redeemer, block.timestamp * 2, _zeroMins());
        vm.prank(redeemer);
        parallelizer.redeem(REDEEM_HALF, redeemer, block.timestamp * 2, _zeroMins());

        uint256 splitN2Out = eurA.balanceOf(redeemer) - eurABefore_n2;

        // == [ Summary ] ==
        uint256 extraN2 = splitN2Out - singleOut;
        uint256 bpsN2   = (extraN2 * 10_000) / singleOut;

        uint256 extraN5 = splitN5Out - singleOut;
        uint256 bpsN5   = (extraN5 * 10_000) / singleOut;

        console.log("[PoC] ==================== SUMMARY: OVER-PENALIZATION ====================");
        console.log("[PoC] Single call output (eurA):        ", singleOut);
        console.log("[PoC] Split N=2 output (eurA):          ", splitN2Out);
        console.log("[PoC] Split N=5 output (eurA):          ", splitN5Out);
        console.log("[PoC] Over-penalization single vs N=2:  ", extraN2);
        console.log("[PoC]   eurA =                          ", bpsN2);
        console.log("[PoC]   bps of single output");
        console.log("[PoC] Over-penalization single vs N=5:  ", extraN5);
        console.log("[PoC]   eurA =                          ", bpsN5);
        console.log("[PoC]   bps of single output");

        // == [ Assertions ] ==
        assertGt(splitN2Out, singleOut, "Split N=2 must receive more eurA than single call");
        assertGt(splitN5Out, splitN2Out, "Split N=5 must receive more eurA than N=2");

        // Fee rate must be non-increasing across the 5 splits
        for (uint256 i = 1; i < 5; ++i) {
            assertLe(fee_c[i], fee_c[i - 1], "Fee rate must be non-increasing across splits");
        }
    }
}
```

Recommended Mitigation: This observation does not require a code change. The per-call snapshot evaluation is a deliberate design choice and is consistent with the protocol's incentive to penalize early redemptions. The note is for the governance and risk team to be aware of when calibrating the curve:

1. Ascending segment slope is the primary lever: The gap between single-call and path-integrated pricing is proportional to the slope of the ascending segment. The current production curve rises 4.5% (from PF=0.95 to PF=0.995) over a 2% CR range — this is already a steep ascent in relative terms. A shallower slope reduces the over-penalization for large redemptions.
2. Document the invariant: Document that the `penaltyFactor` is evaluated at entry-time CR (not integrated) regardless if the same redemption traverses the penalization curve to a better segment.

Parallel: Fixed in commit 97680d2.

Cyfrin: Verified. This behavior has been documented on the natspec of the `Redeemer::_quoteRedemptionCurve` function.

7.2.7 `TokenP` dual inheritance from `ERC20PermitUpgradeable` and `EIP3009` creates independent EIP-712 version sources - `VERSION_HASH` constant diverges from storage on a version-bumping upgrade, breaking `permit()`

Description: `TokenP` is a UUPS upgradeable ERC-20 that inherits from two parent contracts that each provide a complete, independent EIP-712 domain separator implementation: `ERC20PermitUpgradeable` (via OZ v5's `EIP712Upgradeable`) for EIP-2612 `ERC20PermitUpgradeable::permit`, and the custom `EIP3009` abstract contract for gasless `transferWithAuthorization` / `receiveWithAuthorization` / `cancelAuthorization`. Because `DOMAIN_SEPARATOR` must resolve to a single function, `TokenP` overrides it to call `EIP3009::_domainSeparator`:

```
// TokenP.sol:99-101
function DOMAIN_SEPARATOR() external view override(ERC20PermitUpgradeable, EIP3009, IERC20Permit)
↪  returns (bytes32) {
    return _domainSeparator();
}
```

The two paths compute their version component from **different sources**. `ERC20PermitUpgradeable::permit` hashes the struct through OZ's `_hashTypedDataV4`, which calls `_domainSeparatorV4` → `_buildDomainSeparator`, reading the version dynamically from `EIP712Upgradeable` storage:

```
// EIP712Upgradeable.sol (OZ v5.4.0)
function _buildDomainSeparator() private view returns (bytes32) {
    return keccak256(abi.encode(TYPE_HASH, _EIP712NameHash(), _EIP712VersionHash(), block.chainid,
    ↪  address(this)));
}

function _EIP712VersionHash() internal view returns (bytes32) {
    string memory version = _EIP712Version(); // reads EIP712Storage.$._version
    if (bytes(version).length > 0) {
        return keccak256(bytes(version));
    } else {
        bytes32 hashedVersion = $._hashedVersion;
        return hashedVersion != 0 ? hashedVersion : keccak256("");
    }
}
```

`EIP3009::_domainSeparator` instead uses a private constant compiled into bytecode:

```
// EIP3009.sol:39-41, 231-235
bytes32 private constant VERSION_HASH = keccak256(bytes("1"));

function _domainSeparator() internal view returns (bytes32) {
    return keccak256(
```

```
        abi.encode(EIP712_DOMAIN_TYPEHASH, keccak256(bytes(name())), VERSION_HASH, block.chainid,
        ↪  address(this))
    );
}
```

Caller	Domain separator	Version source
`permit()`	OZ `_domainSeparatorV4()`	`EIP712Storage.$._version` (storage)
EIP-3009 ops, `DOMAIN_SEPARATOR()`	`EIP3009._domainSeparator()`	`VERSION_HASH` (`keccak256("1")`)

Both currently produce identical outputs because both start at version `"1"`. No divergence exists today.

The concern is that `VERSION_HASH` is compiled into bytecode and cannot be updated by writing to storage. `EIP712Storage.$._version` can be updated in any future `reinitializer` via `__EIP712_init_unchained(name_, "2")` — a common upgrade pattern to invalidate old signatures. If that happens, `ERC20PermitUpgradeable::permit` silently begins verifying against the version-2 domain while `EIP3009::_domainSeparator` and the public `DOMAIN_SEPARATOR` (getter remains anchored to version `"1"`. Every wallet or aggregator that calls `DOMAIN_SEPARATOR` to construct a `permit` signature receives the wrong domain and gets a `ERC2612InvalidSigner` revert.

Recommended Mitigation: Acknowledge and document this coupling in the upgrade runbook: any future upgrade that bumps `EIP712Storage.$._version` via `__EIP712_init_unchained` must also deploy a new implementation with a matching `VERSION_HASH` constant (e.g., `keccak256(bytes("2"))`). Failing to update one of the two will silently break either `ERC20PermitUpgradeable::permit` or `EIP-3009` signature verification.

Parallel: Fixed in commit ed47f03d.

Cyfrin: Verified. `VERSION_HASH/EIP712` storage version coupling has been documented for future upgrades.